

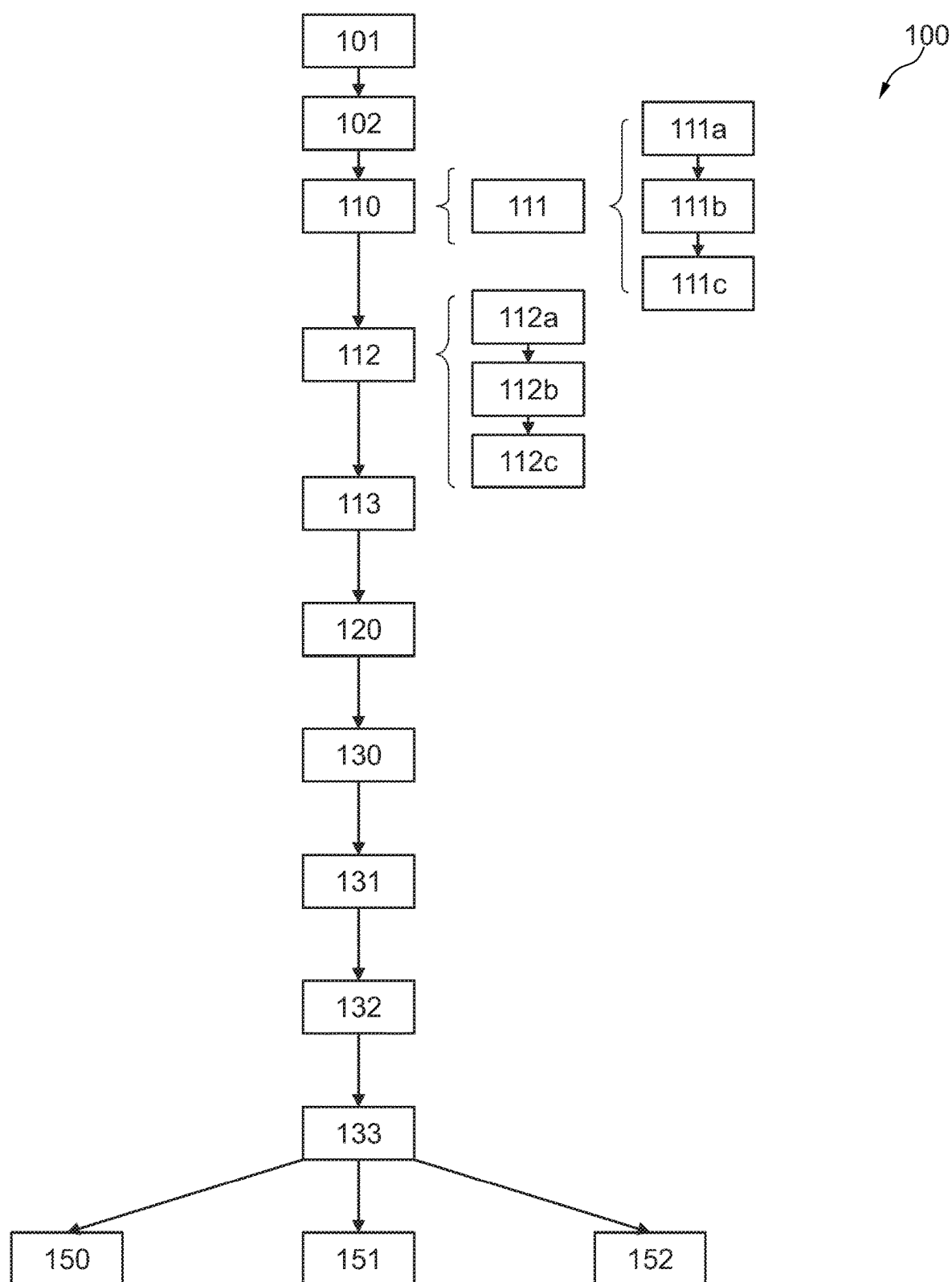


Fig. 1

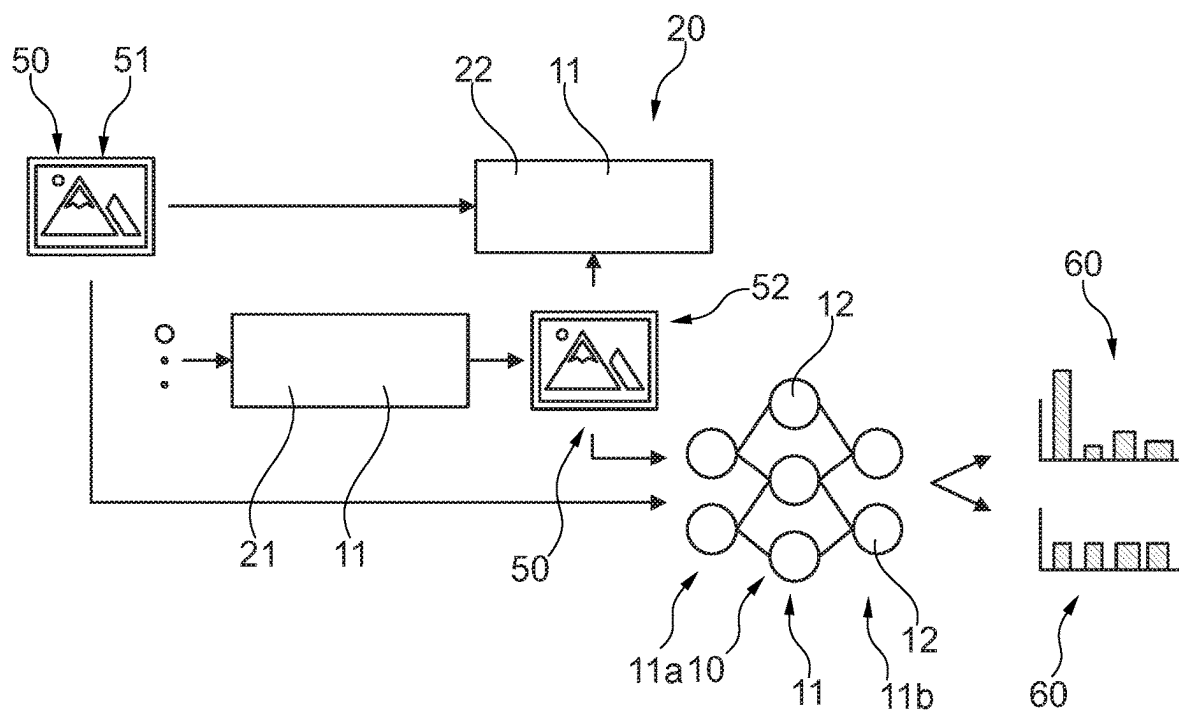


Fig. 2

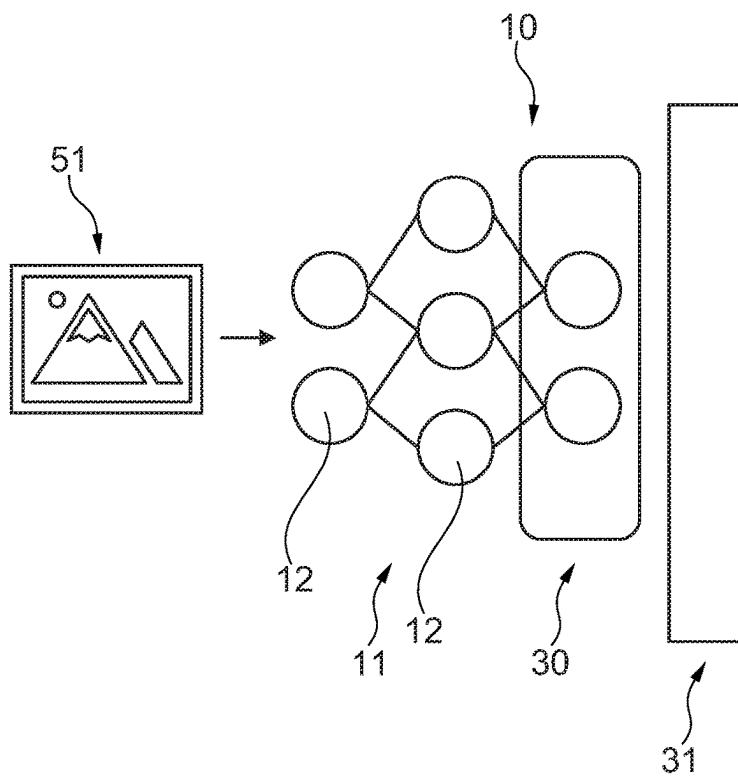


Fig. 3

METHOD AND DEVICE FOR CLASSIFYING SENSOR DATA

[0001] The present invention relates to a method and a device for classifying sensor data according to the independent claims.

PRIOR ART

[0002] In particular in the driverless navigation of vehicles, a classification of objects detected by means of sensors, such as for example road users, is essential, in order to be able to navigate on the basis thereof.

[0003] From the prior art, it is known that at least one previously defined class is assigned to a detected object. However, it is equally crucial that it is detected when an object does not fall within the predetermined classes. Only then can it be guaranteed that a decision with regard to the controlling of the vehicle is not based on false assumptions.

[0004] This problem is all the more important when neural networks are used for the classification. With regard to neural networks, it is known that these only function well for the evaluation of data which were presented to the neural network during the training at least in a similar manner, i.e. when these data follow the distribution of the training data.

PRESENTATION OF THE INVENTION: PROBLEM, SOLUTION, ADVANTAGES

[0005] It is therefore an object of the invention to provide a method for classifying sensor data such that it can be reliably estimated whether an unknown scenario is present, which is unfamiliar to a neural network used for the classification.

[0006] The above-mentioned object is solved by a method for classifying sensor data, which comprises providing a respective averaged feature vector for a multiplicity of classes. For each class, its own averaged feature vector is thus provided.

[0007] The method uses a classifier, wherein the classifier is a neural network trained on the basis of training data. The multiplicity of classes is in particular a finalized and predefined number of classes, on the basis of which the neural network was previously trained. The averaged feature vector is in other words an average vector which is determined for each previously defined class.

[0008] The method here comprises a determining of a feature vector on the basis of sensor data by means of the classifier.

[0009] The method further comprises a respective determining of a cosine similarity, by means of the classifier, between the previously determined feature vector and a respective averaged feature vector for the multiplicity of classes, comparing the respective cosine similarity with a threshold value determined for each class beforehand, and identifying a scenario not represented by the multiplicity of classes if the threshold values for all classes are not reached. The method is thus configured to detect anomalies in the classification of sensor data.

[0010] The cosine similarity constitutes in particular a numerical value which states how similar vectors are, wherein the number lies between +1 and -1. When the cosine similarity is, for example, +1 for a class, the corresponding feature vector corresponds exactly to the averaged feature vector of the corresponding class.

[0011] The neural network was trained beforehand on the basis of training data. These training data comprise the multiplicity of classes. In particular, each class represents an object category, for example a pedestrian or a bicycle. In particular, the training data for each class comprise a multiplicity of training samples, wherein the training samples constitute the object, represented by the class, in as many and different scenarios as possible. Each training sample thus comprises an object in an environmental condition. The training samples preferably concern image data or lidar data, in particular point clouds. Of course, not all possible scenarios can be covered there.

[0012] A scenario is preferably defined by the comprised object and the environmental conditions, for example the weather condition. A scenario can constitute, especially, an object in combination in a weather condition, such as for example snow, rain, fog, dust, hail, in particular as a function of the intensity of the weather condition, for example the density of the fog. A pedestrian in normal visibility (for example with a cloudy sky) constitutes a different scenario to a pedestrian in a hailstorm. Both, however, fall within the "pedestrian" class, wherein the scenario "pedestrian in hailstorm" possibly was not included in the training data because this occurs extremely rarely. A method for object detection is to detect an object in as far as possible all possible environmental conditions, however is also to know its limits, in the sense that the method detects when it can not detect an object on the basis of the present scenario with sufficient confidence, as it was not represented in the training data.

[0013] A scenario which is not represented in the training data, in other words an unknown or unfamiliar scenario, can thus constitute a hitherto not detected, i.e. new, object category and/or a hitherto not included, i.e. new, environmental condition, in particular weather condition, of an object category included in the training data. When the threshold value of at least one class is exceeded, the method can comprise the assigning of the corresponding class and preferably a confidence. In particular, the cosine similarity can be assigned as confidence.

[0014] In the case of not reaching the threshold values of cosine similarity for all classes, the method is able to detect that a scenario is present which was not covered by the multiplicity of classes. The neural network is thus able to detect a deviation of the results which are to be expected, which are based on the training thereof. In other words, it is made possible that a decision is made as to whether the classification was valid or not, which can have a considerable effect on the further processing of the information for the driverless navigation of vehicles.

[0015] The method can thus be configured to detect a scenario of a particular class in the sensor data, however in the case of not reaching all threshold values, to detect that the scenario is not sufficiently represented in the training data. Furthermore, special cases can also be present, for example a pothole or an accident site with vehicle parts on the roadway, wherein the method does not recognize these as represented by the training data, as they constitute an actual new class, in other words object category. Also for example a scenario with a new object (for example a new road user), which was not included in the training data, is also a scenario not represented in the training data, as the entire class is not represented. An example would be a cargo bike, which indeed resembles a bicycle, but nevertheless

constitutes a class of its own. In all the above-mentioned cases, it is established that the detected scenario is not represented in the training data. In the case of a scenario which is not represented by the multiplicity of classes, the concern is therefore with an out-of-distribution sample or respectively an out-of-distribution detection.

[0016] The method therefore differs from methods known from the prior art, which assign a classification to each scenario on the basis of similarity with the classes. The present method can thus also be used in safety-critical applications, such as preferably the driverless navigating of a vehicle, as the classifier is configured to detect unknown scenarios as such in a safety-critical manner. This thus enables not only an error prevention during the application, but also the detection of new scenarios, so that these can be included increasingly into the training data and thus a covering of all occurring scenarios can be guaranteed.

[0017] Furthermore, the method can comprise a determining of a Mahalanobis distance between a covariance matrix, the averaged feature vector and the determined feature vector on the basis of the sensor data for each class. This serves in particular to better detect scenarios not taken into consideration by the multiplicity of classes, as the cosine similarity does not take into consideration the distribution of the features, but only the averaged values thereof. For this reason, the Mahalanobis distance can be used in addition.

[0018] As the Mahalanobis distance is not “bounded”, i.e. not limited, which is necessary in order to estimate this with respect to confidences, a reference value, in particular the function value of the cumulative density function, based on a χ^2 -distribution, is determined. This function, as is known that the Mahalanobis distance follows the χ^2 -distribution, when the number of degrees of freedom corresponds to the dimension of the features. The reference value for each class is determined by the function value of the χ^2 -distribution being determined at the place of the respective Mahalanobis distance. In other words, the Mahalanobis distance of the class is the X-value, which is inserted into the χ^2 -distribution. The corresponding Y-value is subtracted from a one in order to obtain the reference value for the class. A comparison can take place again of the respective reference values with a previously established threshold value for each class. On falling below the threshold values for all classes, a scenario is detected which is not represented by the multiplicity of classes. This double control by means of cosine similarity and the Mahalanobis distance serves for a particularly high safeguard that scenarios which are not taken into consideration in the multiplicity of classes, are also detected.

[0019] In particular on falling below the threshold values for all classes, the method can label, on other words characterize, the detected scenario as unknown compared to the training data.

[0020] Furthermore, a previously defined minimum confidence can be assigned to the detected scenario. In particular, a minimum confidence can be assigned to each class. In particular, the minimum previously defined confidence lies distinctly under the corresponding cosine similarity for each class. This minimum confidence can correspond to zero, for example.

[0021] The method can comprise a storing of sensor data, in which the not represented scenario was detected, as supplement to the training data. In this way, the training data

which were used for the training of the neural network can be supplemented. Thus, sensor data can be stored to new scenarios in an automated manner. Thereby, the laborious manual sifting of scenarios in the “entering” of training data, which is not only time-consuming and thus expensive, but also prone to error, is replaced. In the data recording for training data, hours of sensor data must be recorded and evaluated or respectively labelled manually, in order to guarantee the security of a method for classification. Here, of course, “regular” scenarios are recorded more frequently than, for example, scenarios with very rare weather phenomena or with rare road users. The present method makes it possible to store such sensor data of scenarios not included in the training data and preferably, in a next step, for example in a subsequent “update” of the classifier, to use them in order to also train the latter with regard to the new scenarios. Previously, a manual labelling of the stored sensor data can take place, in order to assign a new class on the basis of the scenario which is unknown to the training data.

[0022] The sensor data can concern, in particular, lidar data. In other words, the sensor data concern in particular a point cloud. The method can comprise, in particular, the receiving of the sensor data by means of a corresponding sensor, in particular a lidar sensor, preferably a solid state lidar sensor. Furthermore, the sensor data can concern radar data or image data. The method here can comprise a pre-processing of the sensor data. In particular, a unit for pre-processing can detect in a point cloud points which clearly take place from the reflection on a shared object. These cohesive points, in other words clusters, can be passed on to the classifier as pre-processed sensor data.

[0023] The neural network of the classifier can be divided in particular functionally into a first unit and a second unit. Here, the first unit and the second unit comprise different layers of the neural network, wherein the differentiation according to first unit and second unit takes place with regard to their function. The first unit is configured as a feature extractor for the extraction of features from the sensor data. The second unit is configured as a classification unit. The first unit serves for the preparation of the sensor data for the second unit.

[0024] In particular, the feature extractor comprises a “multi layer perceptron” (MLP-) network, which comprises at least one “multi layer perceptron” as a layer (MLP layer). Each layer contains a multiplicity of artificial neurons. In particular, the MLP network comprises a multiplicity of MLP layers. The MLP layers are preferably all activated respectively by a non-linear function, in particular a so-called ReLU function. A ReLU function is a function which as Y value assigns zero to a negative x-value, while it otherwise represents a linear function with slope 1. The MLP preferably has at least 3, in particular at least 5, layers, which can comprise several sub-layers.

[0025] Furthermore, the MLP network can comprise at least one fully connected layer, which can also be activated by a non-linear function, in particular a so-called ReLU function.

[0026] The MLP network is able to process input in the form of point clouds. Output of the network are therefore features, in particular in the form of vectors. Here, the last layer in particular has no activation. Finally, a softmax normalization can take place. As a result—before the softmax normalization, preferably a bottleneck vector is obtained. The bottleneck vector is thus the result of the last

layer of the neural network before a possible softmax normalization, and constitutes a feature vector.

[0027] Alternatively, the feature extractor can also be a CNN (convolutional neural network). For this, the point cloud which is present as sensor data must, however, be previously transformed into an image representation. The CNN comprises in particular a multiplicity of layers. In particular, the CNN comprises alternately a convolutional layer and pooling layer. The CNN is configured to extract lines, L features and/or circle features. A feature can also be designated as “feature”.

[0028] The second unit of the classifier comprises at least one, preferably two or three, fully connected layers, which directly adjoin the last layer of the first unit. Preferably, the second unit consists of the at least one fully connected layer. Each layer comprises a multiplicity of artificial neurons. The second unit serves for the processing of the feature vectors of the first unit.

[0029] In particular, the method comprises a training of the classifier, and namely of the neural network of the classifier, by means of the training data, wherein the training data comprise first training samples and second training samples. Here, the second training samples are produced by means of a generative adversarial network (GAN), having a generator and a discriminator, whereas the first training samples are based on a reception of scenarios by means of a sensor. The first training data thus constitute the typical training data, which are entered by means of corresponding sensors, i.e. were received in actual situations. The first training samples thus comprise an in particularly manually assigned label.

[0030] The classifier is carried out in particular on the basis of a cross entropy loss on the basis of the first training samples added with a Kullback-Leibler (KL) divergence on the basis of the second training samples. The KL divergence calculates the difference between two distributions and by addition of the divergence to a loss it is achieved that the distributions equalize accordingly. For this, the KL divergence is added to the loss in an additive manner. The aim is that the classifier detects objects from the second training samples with an equally distributed probability for all classes, so that in the case of application the cosine similarity thereof with all averaged feature vectors of the first training samples lies below the corresponding threshold values.

[0031] The training takes place on the basis of the manual labels of the first training samples and the labels produced by means of the classifier by a corresponding comparison by means of the cross entropy loss and the additively added KL divergence and an adapting of the weights of the neural network of the classifier. As in the training the averaged feature vectors are not present, in the training work is carried out on the basis of uncertainties which the softmax normalization outputs. The aim in the training is that the uncertainties in second training samples are distributed equally to all classes, so that then in the case of application, i.e. after completed training, in a scenario which is not represented in the training data, the cosine similarities lie under the threshold values and are thus identified as such.

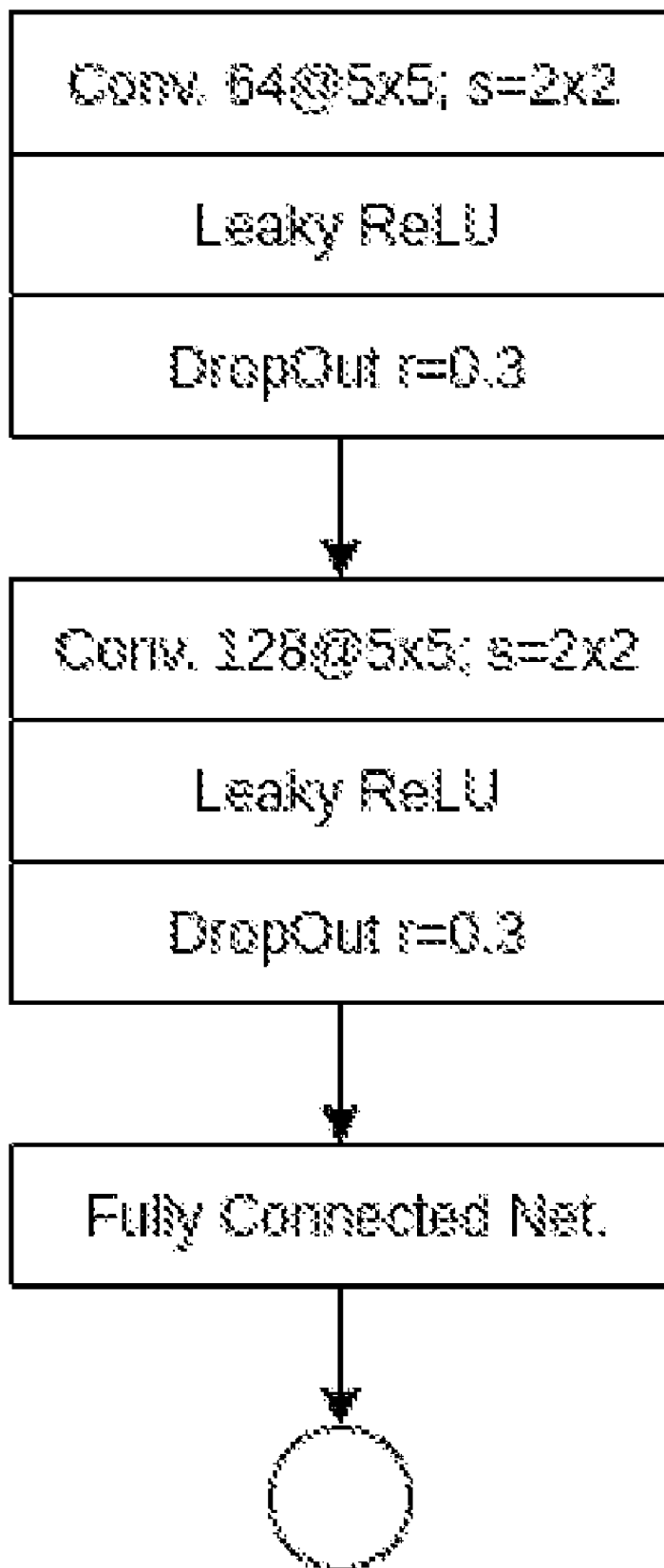
[0032] The generator and also the discriminator of the GAN can concern artificial neural networks. Here, the generator generates so-called candidates, second training samples here, which the discriminator evaluates. The aim of the generator is to learn to generate results according to a particular distribution. The discriminator on the other hand is trained to differentiate the results of the generator from the data of the genuine, predetermined distribution. The target function of the generator consists in generating results which the discriminator can not differentiate. In this way, second training samples are produced, which lie at the decision boundaries of the classifier. The classifier evaluates the results with an evenly distributed probability. Therefore, the KL divergence is also added to the generator loss in an additive manner.

[0033] The method comprises in particular a training of the GAN, especially of the generator and of the discriminator, wherein the training of the classifier and of the GAN takes place jointly. Furthermore, the training of the feature extractor can also take place jointly.

[0034] The training of the generator is carried out on the basis of a binary cross entropy loss on the basis of the second training samples, added with a KL divergence on the basis of the second training samples. The second training samples are evaluated by means of the discriminator. Here, the discriminator evaluates originally first training samples for example with a one, whereas second training samples are evaluated differently, for example with a zero. The binary cross entropy loss refers to the evaluation of the second training samples by the discriminator. The KL divergence is based on a classification of the second training samples by the classifier compared to an evenly distributed probability. The weights of the generator in the training are adapted so that it generates second training samples which the discriminator evaluates just as first training samples, however the classifier assigns an evenly distributed probability to all classes in a distributed manner.

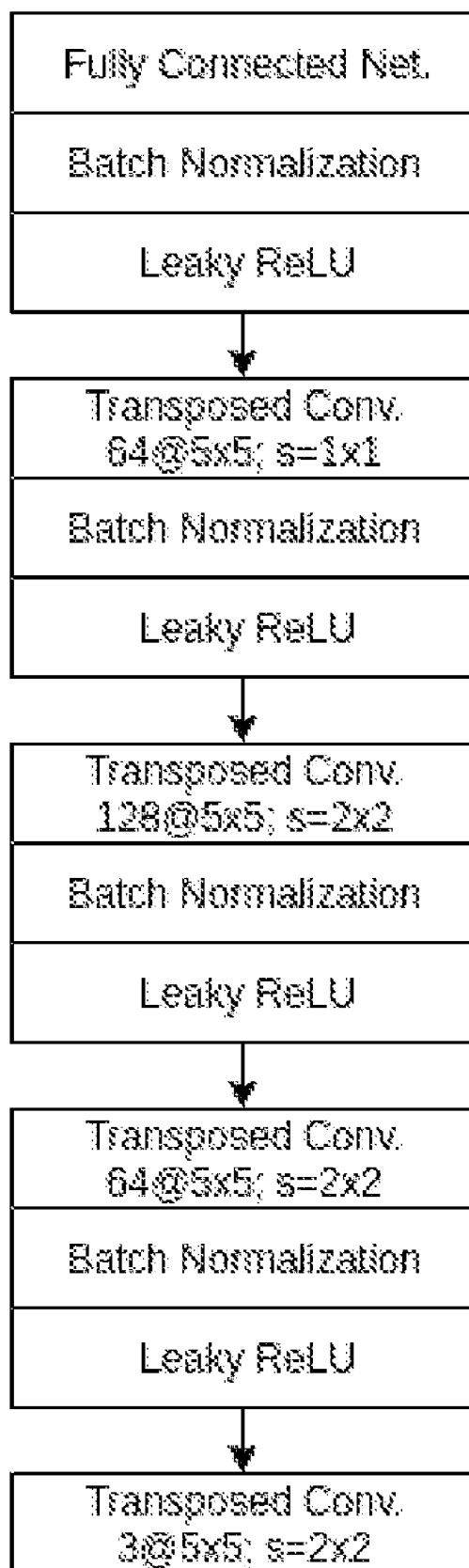
[0035] The discriminator is also trained on the basis of a cross entropy loss, in particular binary cross entropy loss, and namely on the basis of the first training samples and the second training samples generated by the generator. The joint training of the classifier and of the GAN serves for the generator not only to attempt to deceive the discriminator, but also to produce unknown scenarios for the classifier, which this evaluates with an evenly distributed probability. Thereby, the reliability of the detecting of unknown scenarios is substantially increased. Here also, during the training in particular the weights of the neural networks are adapted.

[0036] The discriminator can have in particular at least one, preferably two, convolutional layers, and are activated by a Relu function, especially a leaky ReLU function. A leaky ReLU function is a function which assigns to an x-value as Y value a small value greater than zero (e.g. 0.01x), whereas it otherwise constitutes a linear function with slope 1. Lastly, preferably a fully connected layer with a single output neuron is used. In particular, the discriminator is configured as follows:



[0037] The generator can consist of a fully connected layer with a batch normalization and an activation by means of a “leaky ReLU”. Subsequently, three transposed convolutional blocks can follow. In particular, respectively with 5×5 kernel and with a batch normalization and a leaky ReLU activation. The last layer can follow from a transposed convolutional layer, preferably again with 5×5 kernel.

[0038] In particular, the generator is configured as follows:



[0039] The training can take place in particular as follows:

[0040] Firstly, the loss of the discriminator, namely the cross entropy loss, is generated, in order to differentiate first training data and second training data which originate from the generator. The discriminator loss is thus defined as the sum of the binary cross entropy loss of the first training samples and of the generated second training samples. D_n stands here for a first training sample. “noise” is a 100 dimensional random vector generated from a standard Gaussian distribution (mean=0, stddev=1). This random vector is input for the generator.

$$\text{Loss}_d = BCE(\text{disc}(D_n), 1) + BCE(\text{disc}(\text{gen}(\text{noise})), 0)$$

[0041] The generator is not only trained to deceive the discriminator, but also to generate second training samples, and namely at the decision boundaries of the classifier. The loss of the generator is thus defined as:

$$\text{Loss}_g = BCE(\text{disc}(\text{gen}(\text{noise})), 1) + KL\left[\text{cls}(\text{gen}(\text{noise})) \parallel \frac{1}{|c|}\right],$$

wherein KL stands for the KL divergence, $|c|$ represents the number of the classes or respectively the quantity of the classes and $1/|c|$ represents the uniformly distributed probability over the classes. In the manner described above, the corresponding weights of the neural networks are adapted in order to deceive the discriminator that the second training samples follow the distribution of the first training samples and furthermore to achieve an equal distribution of the probabilities of the classifier to second training samples. This is important, because it is then ensured that an unknown scenario falls under all threshold values of the multiplicity of classes.

[0042] The loss of the classifier is defined as:

$$\text{Loss}_{cls} = CE(\text{cls}(D_n), \text{labels}) + KL\left[\text{cls}(\text{gen}(\text{noise})) \parallel \frac{1}{|c|}\right].$$

[0043] All weights are initialized with the Xavier uniform initializer.

[0044] Advantageously, an averaged feature vector is determined for each class, wherein the training data comprises a number of first training samples per class and wherein the averaged feature vector of a class is determined by the following steps:

[0045] Firstly, in particular by means of the classifier (before the softmax normalization) a feature vector is determined for each first training sample of a class. This concerns here in particular the bottleneck vector. These feature vectors are added to obtain an added feature vector. The latter is divided by the number of the first training samples of the corresponding class. As a result, the averaged feature vector of a class is obtained. The averaged feature vector concerns in particular the averaged bottleneck vector of the corresponding class. The averaged feature vectors are determined in particular after a training of the classifier.

[0046] For each class in particular the threshold value is determined, wherein the threshold value of a class is determined by the following steps: Firstly the cosine similarity of

the feature vectors of each first training sample is determined with the averaged feature vector of the class. The determined cosine similarities are now either averaged and thus an averaged cosine similarity is calculated, or the smallest cosine similarity is determined from the quantity of cosine similarities of the first training data of the class. The threshold value can be established either as the averaged cosine similarity or the smallest cosine similarity.

[0047] For example, the threshold can have been established at 0.85. If a cosine similarity between a determined feature vector and the averaged feature vector of 0.92 is now determined, the corresponding class can be issued with a high confidence and thus a valid detection can be confirmed.

[0048] In particular, the present method for classifying sensor data is part of a method for the driverless navigation of a vehicle.

[0049] In a further aspect, the invention comprises a device for classifying sensor data, wherein the device comprises a classifier and is configured to perform a method described above. In particular, the classifier is trained as described above. In addition, the device can comprise a GAN described above, which in particular is trained as described above. The classifier can comprise in particular a decision module, which on the basis of the threshold values and the determined cosine similarities and preferably Mahalanobis distances assigns a class and/or a confidence and/or a label as unknown. In particular, the device serves for the navigation of a driverless vehicle.

[0050] The invention further comprises a computer program product which comprises a computer-readable storage medium on which a program is stored, which enables a computer, after it has been loaded into the memory of the computer, to carry out a method, described above, for classifying sensor data in cooperation with a device, described above.

[0051] In addition, the invention relates to a computer-readable storage medium on which a program is stored, which enables a computer, after it has been loaded into the memory of the computer, to perform a method, described above, for classifying sensor data in cooperation with a device, described above.

BRIEF DESCRIPTION OF THE FIGURES

[0052] There are shown in purely schematic representation

[0053] FIG. 1: a method diagram of a method according to the invention,

[0054] FIG. 2: a training of a classifier and of a GAN, and

[0055] FIG. 3: a determining of an averaged feature vector.

PREFERRED EMBODIMENTS OF THE INVENTION

[0056] FIG. 1 shows a method diagram of a method 100 according to the invention for classifying objects. The method 100 comprises providing 110 a respective averaged feature vector 30 for a multiplicity of classes. By means of a classifier 10 a feature vector is determined on the basis of sensor data 120. A cosine similarity between the determined feature vector and the respective averaged feature vectors is determined for all classes 130 and the cosine similarities are compared to a previously determined threshold value 131.

On falling below the threshold values for all classes, a scenario not represented by the multiplicity of the classes is detected **133**.

[0057] The method **100** can comprise in particular the training **101** of the classifier **10** and the training **102** of the GAN **20**. This takes place in particular jointly.

[0058] Furthermore, the respective averaged feature vectors **30** can be determined for the multiplicity of classes **111**. For this, in particular a feature vector is determined for each first training sample of a respective class **111a**, the corresponding feature vectors of all first training samples for the respective class are added **111b** and divided by the number of the first training samples of the class **111c**.

[0059] Furthermore, the respective threshold value can be determined for the multiplicity of classes **112**. This takes place in particular by determining **112a** the cosine similarities of the feature vector of each training sample with the averaged feature vector **30** of the class. The averaged cosine similarity or the smallest cosine similarity is determined **112b**. The averaged cosine similarity is calculated in particular by adding the cosine similarities of all first training samples and through division by the number of the first training samples. The determining of the smallest cosine similarity takes place simply by establishing the smallest value of the cosine similarities. The averaged cosine similarity or the smallest cosine similarity is established as threshold value **112c**.

[0060] As a further parameter for establishing whether or not the sensor data comprise a scenario represented in the training data, a Mahalanobis distance can be determined between a covariance matrix, the averaged feature vector and the determined feature vector **132**. Based on the Mahalanobis distance and the χ^2 -distribution, for each class a reference value can be determined, which is compared with the respective threshold value of the class. The covariance matrix for each class can be determined beforehand **113**.

[0061] If the threshold values for all classes are not reached, the detected scenario can be labelled as unknown compared to the training data **150**, a minimum confidence can be assigned to the scenario **151** and/or the corresponding sensor data can be stored as a supplement to the training data **152**.

[0062] FIG. 2 shows the training of a classifier **10** and of a GAN **20**, wherein the GAN **20** comprises a generator **21** and a discriminator **22**. As input for the discriminator **22**, first training samples **51** are provided, and second training samples **52** by the generator **21**. The generator **21** generates second training samples **51** at the decision boundaries of the classifier **10** and thus attempts to deceive the discriminator **22** and to allow the classifier **10** to assign an evenly distributed probability to the second training samples **52**.

[0063] Both the classifier **10** and also the generator **21** and the discriminator **22** concern neural networks **11**. In the case of the classifier **10** by way of example neurons **12**, which are arranged in layers, are illustrated. The neural network **11** of the classifier **10** can be divided functionally into two units, a first unit **11a** and a second unit **11b**. The first unit **11a** is configured as a feature extractor, whereas the second unit **11b** serves as a classification unit.

[0064] On the right adjacent to the classifier **10**, determined uncertainties **60** of the softmax normalization **31** are represented divided in tabular form to the multiplicity of classes. In the upper part, the uncertainties **60** of the softmax

normalization **31** is shown for a first training sample, wherein the first class is by far the highest, so that at least this class is assigned with a high confidence. As in the training no averaged feature vector is present yet, operations are carried with the uncertainties obtained after the softmax normalization.

[0065] It is different in the lower example, which shows a second training sample in which the uncertainties **60** of the softmax normalization **31** are evenly distributed for all classes.

[0066] FIG. 3 represents how, after the training, the classifier **10** an averaged vector **30** is determined, and namely before a softmax normalization **31**. This takes place in the last layer of the second unit, therefore of the classifier, before the softmax normalization **31**, and namely on the basis of all first training samples **51**, accordingly for each class. In the case of application, then on the basis of sensor data and the averaged feature vectors **30** for the classes the cosine similarity is determined according to FIG. 1.

EXAMPLES OF THE NEURAL NETWORKS

[0067] The feature extractor can be constructed as follows:

Embodiment as MLP

[0068] Input: Batch size×2048×3

[0069] Layer 0:

[0070] Layer 0.1: Multi Layer Perceptrons (MLP)

[0071] Number of functions: 64

[0072] Kernel 1×3

[0073] Batch normalization

[0074] Activation function: ReLU

[0075] Layer 0.2: MLPs

[0076] Number of functions: 128

[0077] Kernel 1×1

[0078] Batch normalization

[0079] Activation function: ReLU

[0080] Layer 0.3: MLPs

[0081] Number of functions: 1024

[0082] Kernel 1×1

[0083] Batch normalization

[0084] Activation function: ReLU

[0085] Layer 0.4: Fully Connected

[0086] Output: 512

[0087] Batch normalization

[0088] Activation function: ReLU

[0089] Layer 0.5: Fully Connected

[0090] Output: 256

[0091] Batch normalization

[0092] Activation function: ReLU

[0093] Layer 0.5: Fully Connected

[0094] Output: 9

[0095] No activation function

[0096] Initializing of the variables with 0

[0097] Layer 0.6: Addition with unit matrix $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

[0098] Output: Batch size×3×3

[0099] Layer 1: Input*Output Layer 0

[0100] Layer 2: MLPs

[0101] Number of functions: 64

[0102] Kernel 1×3

[0103] Batch normalization

[0104] Activation function: ReLU

- [0105] Layer 3: MLPs
 - [0106] Number of functions: 64
 - [0107] Kernel 1×1
 - [0108] Batch normalization
 - [0109] Activation function: ReLU
- [0110] Layer 4: Learning of matrix for multiplication
 - [0111] Layer 4.1: MLP
 - [0112] Number of functions: 64
 - [0113] Kernel 1×1
 - [0114] Batch normalization
 - [0115] Activation function: ReLU
 - [0116] Layer 4.2: MLPs
 - [0117] Number of functions: 128
 - [0118] Kernel 1×1
 - [0119] Batch normalization
 - [0120] Activation function: ReLU
 - [0121] Layer 4.3: MLPs
 - [0122] Number of functions: 1024
 - [0123] Kernel 1×1
 - [0124] Batch normalization
 - [0125] Activation function: ReLU
 - [0126] Layer 4.4: Fully Connected
 - [0127] Output: 512
 - [0128] Batch normalization
 - [0129] Activation function: ReLU
 - [0130] Layer 4.5: Fully Connected
 - [0131] Output: 256
 - [0132] Batch normalization
 - [0133] Activation function: ReLU
 - [0134] Layer 4.5: Fully Connected
 - [0135] Output: 4096
 - [0136] No activation function
 - [0137] Initializing of the variables with 0
 - [0138] Layer 4.6: Addition with unit matrix (size 64)
 - [0139] Output: Batch size×64×64
- [0140] Layer 5: Layer 3*Output Layer 4
- [0141] Layer 6: MLPs
 - [0142] Number of functions: 64
 - [0143] Kernel 1×1
 - [0144] Batch normalization
 - [0145] Activation function: ReLU
- [0146] Layer 7: MLPs
 - [0147] Number of functions: 128
 - [0148] Kernel 1×1
 - [0149] Batch normalization
 - [0150] Activation function: ReLU
- [0151] Layer 8: MLPs
 - [0152] Number of functions: 512
 - [0153] Kernel 1×1
 - [0154] Batch normalization
 - [0155] Activation function: ReLU
- [0156] Layer 9: Classification unit
 - [0157] Layer 9.1 Fully Connected Layer:
 - [0158] Number of output neurons: 512
 - [0159] Activation function: ReLU
 - [0160] Layer 9.2: Dropout
 - [0161] Dropout rate: 0.5
 - [0162] Layer 9.3 Fully Connected Layer:
 - [0163] Number of output neurons: 512
 - [0164] Activation function: ReLU
 - [0165] Layer 9.4: Dropout
 - [0166] Dropout rate: 0.5
 - [0167] Layer 9.5: Fully Connected Layer (=bottle-neck vector):
 - [0168] Number of output neurons: number of classes to be classified
- [0169] GGF Softmax normalization layer
 - Embodiment as CNN
- [0170] Input: Batch size×80×128×2 (2 channels for solid state: intensity and depth)
- [0171] Layer 1: Convolution
 - [0172] Kernel size: 3×3
 - [0173] Number of output feature maps: 64
 - [0174] Activation: ReLU
- [0175] Layer 2: Convolution
 - [0176] Kernel size: 3×3
 - [0177] Number of output feature maps: 64
 - [0178] Activation: ReLU
- [0179] Layer 3: Max Pooling
 - [0180] Kernel size: 2×2
- [0181] Layer 4: Convolution
 - [0182] Kernel size: 3×3
 - [0183] Number of output feature maps: 128
 - [0184] Activation: ReLU
- [0185] Layer 5: Convolution
 - [0186] Kernel size: 3×3
 - [0187] Number of output feature maps: 128
 - [0188] Activation: ReLU
- [0189] Layer 6: Max Pooling
 - [0190] Kernel size: 2×2
- [0191] Layer 7: Convolution
 - [0192] Kernel size: 3×3
 - [0193] Number of output feature maps: 256
 - [0194] Activation: ReLU
- [0195] Layer 8: Convolution
 - [0196] Kernel size: 3×3
 - [0197] Number of output feature maps: 256
 - [0198] Activation: ReLU
- [0199] Layer 9: Max Pooling
 - [0200] Kernel size: 2×2
- [0201] Layer 10: Convolution
 - [0202] Kernel size: 3×3
 - [0203] Number of output feature maps: 512
 - [0204] Activation: ReLU
- [0205] Layer 11: Convolution
 - [0206] Kernel size: 3×3
 - [0207] Number of output feature maps: 512
 - [0208] Activation: ReLU
- [0209] Layer 12: Max Pooling
 - [0210] Kernel size: 2×2
- [0211] Layer 13: Convolution
 - [0212] Kernel size: 3×3
 - [0213] Number of output feature maps: 512
 - [0214] Activation: ReLU
- [0215] Layer 14: Convolution
 - [0216] Kernel size: 3×3
 - [0217] Number of output feature maps: 512
 - [0218] Activation: ReLU
- [0219] Layer 15: Max Pooling
 - [0220] Kernel size: 2×2
- [0221] Layer 16: Flattening of the features (preparation, in other words adaption of the dimension, as input for classifier)
- [0222] Layer 17: Classification unit
 - [0223] Layer 17.1 Fully Connected Layer:
 - [0224] Number of output neurons: 512
 - [0225] Activation function: ReLU

- [0226] Layer 17.2: Dropout
- [0227] Dropout rate: 0.5
- [0228] Layer 17.3 Fully Connected Layer:
- [0229] Number of output neurons: 512
- [0230] Activation function: ReLU
- [0231] Layer 17.4: Dropout
- [0232] Dropout rate: 0.5
- [0233] Layer 17.5: Fully Connected Layer (=bottle-neck vector):
- [0234] Number of output neurons: Number of classes to be classified
- [0235] GGF Normalization with softmax normalization
- [0236] The training of the classifier and of the GAN can be determined by the following parameters:
- [0237] Optimizer: Adam Optimizer (respectively own optimizer for discriminator, generator and classifier)
- [0238] Learning Rate: 0.0001
- [0239] Exponential decay: Staircase
- [0240] The learning rate decreases as a step function. After 200000 steps it decreases by half, until it has reached a minimum of 0.00001, after which it remains constant
- [0241] Batch size: 32
- [0242] Initialization of all variables (except initialization with 0): Xavier initialization
- [0243] Total epochs: 100

LIST OF REFERENCE NUMBERS

- [0244] 10 classifier
- [0245] 11 neural network
- [0246] 11a first unit
- [0247] 11b second unit
- [0248] 12 neuron
- [0249] 20 GAN
- [0250] 21 generator
- [0251] 22 discriminator
- [0252] 30 averaged feature vector
- [0253] 31 softmax normalization
- [0254] 50 training data
- [0255] 51 first training samples
- [0256] 52 second training samples
- [0257] 60 uncertainties
- [0258] 100 method
- [0259] 101 training of the classifier
- [0260] 102 training of the GAN
- [0261] 110 providing a respective averaged feature vector for a multiplicity of classes
- [0262] 111 determining a respective averaged feature vector for the multiplicity of classes
- [0263] 111a determining a feature vector for each first training sample of a class
- [0264] 111b adding the feature vectors of all first training samples of a class for obtaining an added feature vector
- [0265] 111c dividing the added feature vector by the number of the first training samples of the class
- [0266] 112 determining a respective threshold value for the multiplicity of classes
- [0267] 112a determining the cosine similarity of the feature vector of each first training sample with the averaged feature vector of the class
- [0268] 112b determining the averaged cosine similarity or the smallest cosine similarity of the first training data of the class

- [0269] 112c establishing the averaged cosine similarity or the smallest cosine similarity as threshold value of the class
- [0270] 113 determining a covariance matrix
- [0271] 120 determining a feature vector on the basis of sensor data
- [0272] 130 determining a cosine similarity between the feature vector and a respective averaged feature vector for the multiplicity of classes
- [0273] 131 comparison of the respective cosine similarity with a threshold value ascertained for each class beforehand
- [0274] 132 determining a Mahalanobis distance between a covariance matrix, the averaged feature vector and the ascertained feature vector
- [0275] 133 detecting a scenario not represented by the multiplicity of classes if the threshold values for all classes are not reached
- [0276] 150 on not reaching the threshold values for all classes, labelling the detected scenario as unknown compared to the training data
- [0277] 151 on not reaching the threshold values for all classes, assigning a previously defined minimum confidence to the detected scenario
- [0278] 152 storing the sensor data as supplement to the training data

1. A method for classifying sensor data, wherein the method comprises providing a respective averaged feature vector for a multiplicity of classes, wherein the method comprises the following steps carried out by means of a classifier, wherein the classifier comprises at least one neural network trained on the a basis of training data:
 - determining a feature vector on the basis of sensor data, respectively determining a cosine similarity between the feature vector and the respective averaged feature vector for the multiplicity of classes,
 - a comparison of the respective cosine similarity with a threshold value established for each class beforehand, and
 - detecting a scenario not represented by the multiplicity of classes if the threshold values are not reached for all classes.
2. The method according to claim 1, wherein the classifier furthermore determines a Mahalanobis distance between a covariance matrix and the averaged feature vector and the determined feature vector for each class.
3. The method according to claim 2, wherein in a reference value for each class is determined on the basis of the Mahalanobis distance and the χ^2 -distribution, wherein the reference value is compared with a threshold value established for each class, wherein the method comprises a detecting of a scenario not represented by the multiplicity of classes if the threshold values for all classes are not reached.
4. The method according to claim 1, wherein if the threshold value for all classes are not reached, the detected scenario is labelled as unknown compared to the training data.

5. The method according to claim 1, wherein if the threshold values for all classes are not reached, a previously defined minimum confidence is assigned to the detected scenario.

6. The method according to claim 1, wherein the method comprises a storing of the sensor data as supplement to the training data.

7. The method according to claim 1, wherein the method comprises a training of the classifier by means of the training data, wherein the training data comprise first training samples and second training samples, wherein the second training samples are produced by means of a generative adversarial network, wherein the GAN comprises a generator and a discriminator, wherein the first training samples are based on a recording of scenarios by means of a sensor.

8. The method according to claim 7, wherein the training of the classifier is carried out on the basis of a cross entropy loss on the basis of the first training samples-added with a Kullback-Leibler divergence on the basis of the second training samples.

9. The method according to claim 7, wherein the method comprises a training of the GAN, namely of the generator and of the discriminator, wherein the training of the classifier and the training of the GAN take place jointly.

10. The method according to claim 9, wherein the training of the generator is carried out on the basis of a binary cross entropy loss on the basis of the second training samples added with a Kullback-Leibler divergence on the basis of the second training samples.

11. The method according to claim 7, wherein the averaged feature vector is determined for each class, wherein the training data comprises a number of first training samples per class, wherein the averaged feature vector of a class is determined by the following steps:
determining a feature vector for each first training sample of a class,
adding the feature vectors of all first training samples of a class for obtaining an added feature vector
dividing the added feature vector by the number of the first training samples of the class.

12. The method according to claim 7, wherein the threshold value is determined for each class, wherein the threshold value of a class is determined by the following steps:
determining the cosine similarity of the feature vector of each first training sample with the averaged feature vector of the class,
determining the averaged cosine similarity or the smallest cosine similarity of the first training data of the class;
establishing the averaged cosine similarity or the smallest cosine similarity as threshold value of the class.

13. A device for classifying sensor data, wherein the device comprises a classifier and is configured to carry out a method for classifying sensor data, wherein the method comprises providing a respective averaged feature vector for a multiplicity of classes, wherein the method comprises the following steps carried out by means of the classifier,
wherein the classifier comprises at least one neural network trained on the basis of training data:
determining a feature vector on the basis of sensor data, respectively determining a cosine similarity between the feature vector and the respective averaged feature vector for the multiplicity of classes,
a comparison of the respective cosine similarity with a threshold value established for each class beforehand, and
detecting a scenario not represented by the multiplicity of classes if the threshold values are not reached for all classes.

14. (canceled)

15. A transitory computer-readable storage medium on which a program is stored which enables a computer, after it has been loaded into the memory of the computer, to carry out a method for classifying sensor data, wherein the method comprises providing a respective averaged feature vector for a multiplicity of classes, wherein the method comprises the following steps carried out by means of the classifier,
wherein the classifier comprises at least one neural network trained on the basis of training data:
determining a feature vector on the basis of sensor data, respectively determining a cosine similarity between the feature vector and the respective averaged feature vector for the multiplicity of classes,
a comparison of the respective cosine similarity with a threshold value established for each class beforehand, and
detecting a scenario not represented by the multiplicity of classes if the threshold values are not reached for all classes.

* * * * *